

Ownership

Course: Rust Basics

Author: Aleksandr Kugushev

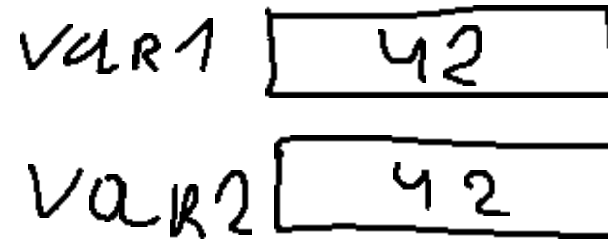
Move/Copy/Borrow Semantics in Programming

Language	Move	Copy	Borrow	Default (Primitives)	Default (Complex Types)	Immutable Parameters
C	-	Y	-	Copy	-	Y (opt-in via 'const')
C#	-	Y	Y	Copy	Mutable Borrow	-
Java	-	Y	Y	Copy	Mutable Borrow	Y (opt-in via 'final')
Rust	Y	Y	Y	Copy	Move/Copy	Y (opt-out via 'mut')

<https://gist.github.com/dubrowgn/c61372647160ae4a52f65ef91da99ddc>

Copy

```
let var1 = 42;  
let var2 = var1;
```



```
pub trait Copy: Clone {  
    // Empty.  
}
```

Copy trait

- Simple bit-wise copy
- Not overloadable
- Everything which is Copy must also implement Clone
- Can't implement Copy for &mut
- Shouldn't implement Copy for mutable structures

Why copy is not a default behavior?

- Smart Pointers
- Optimizations
- Missing Updates

```
fn update_int(mut str: MyStruct)
{
    str.integer = 123;
}
```

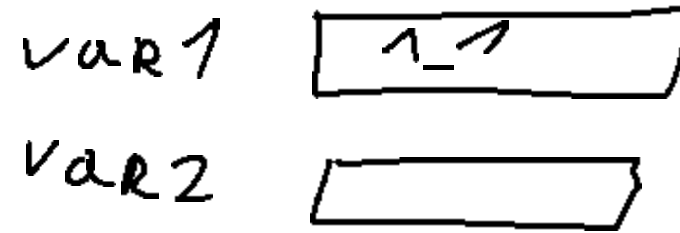
```
#[derive(Copy, Clone)]
struct MyStruct {
    integer: i32,
}
```

```
let str = MyStruct { integer: 42 };
update_int(str);
println!("{}", str.integer); // ???
```

>> 42

Move

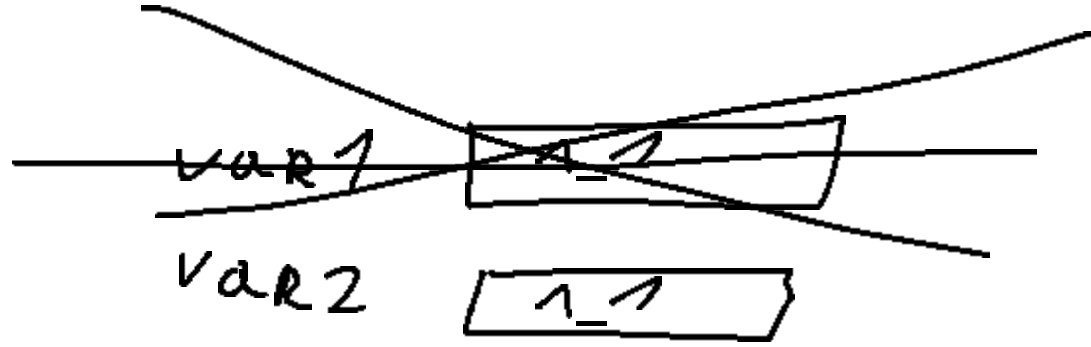
```
let var1 = my_struct;  
let var2 = var1;
```



memcpy (!)

Move

```
let var1 = my_struct;  
let var2 = var1;  
println!("{}", var1);
```



error[E0382]: borrow of moved value: `var1`:

```
let var2 = var1;  
    ---- value moved here
```

```
println!("{}", var1);  
          ^^^^ value borrowed here after move
```

Move to method

```
let var1 = my_struct;  
my_fn(var1);  
println!("{}", var1);
```

error[E0382]:

```
my_fn(var1);
```

---- value moved here

```
println!("{}", var1);
```

^^^^ value borrowed here after move

Drop

```
pub fn drop<T>(_x: T) {}
```

Drop on move out of scope

```
let var1 = my_struct1; ←  
{  
  let var2 = my_struct2;  
}
```

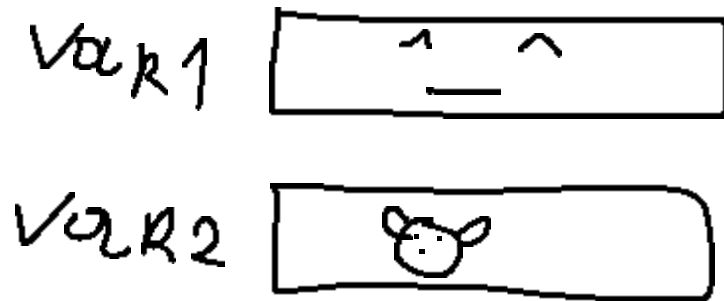
var1 ^
_
^

```
let var1 = my_struct1; ←  
{  
  let var2 = var1;  
}
```

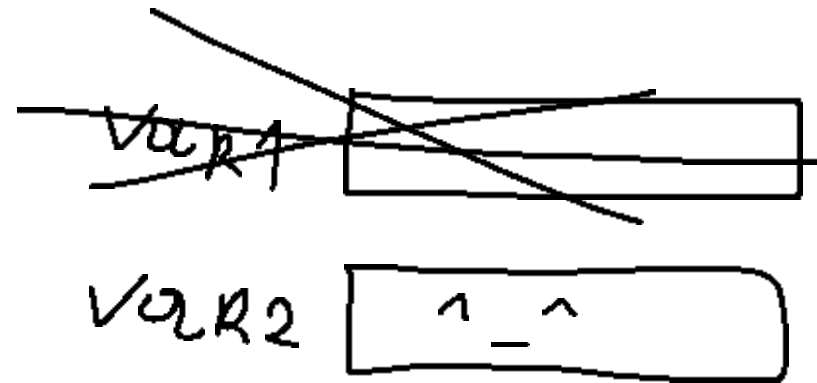
var1 ^
_
^

Drop on move out of scope

```
let var1 = my_struct1;  
{  
  let var2 = my_struct2;  
}
```

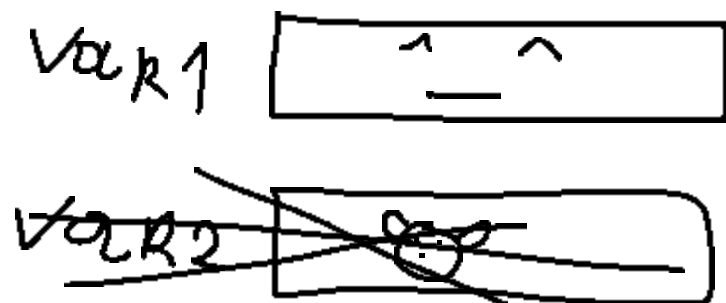


```
let var1 = my_struct;  
{  
  let var2 = var1;  
}
```

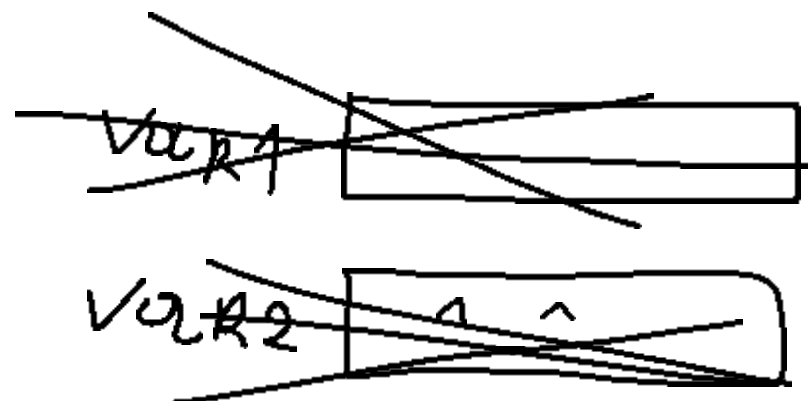


Drop on move out of scope

```
let var1 = my_struct1;  
{  
  let var2 = my_struct2;  
} ←
```



```
let var1 = my_struct;  
{  
  let var2 = var1;  
} ←
```

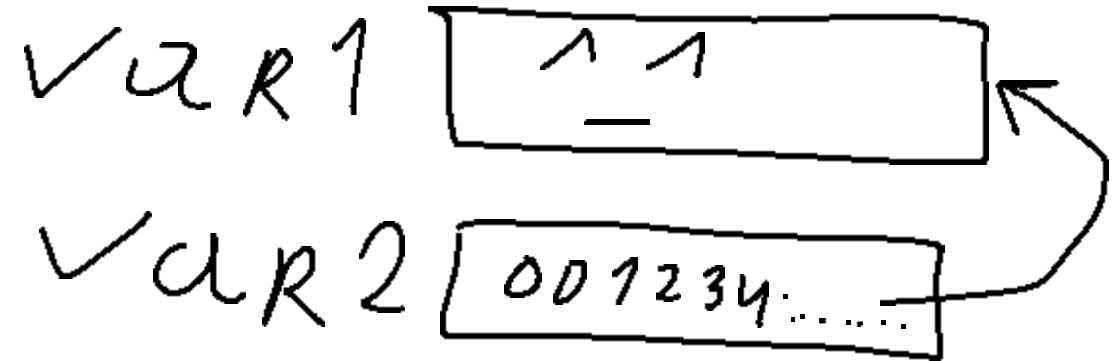


What happens with variable after move

```
let mut str = StrWithInt { int1: 42 };  
let ptr: *const StrWithInt = &str as *const StrWithInt;  
unsafe { println!("{}", (*ptr).int1); } // 42  
drop(str);  
unsafe { println!("{}", (*ptr).int1); } // 42  
str = StrWithInt { int1: 1 };  
unsafe { println!("{}", (*ptr).int1); } // 1
```

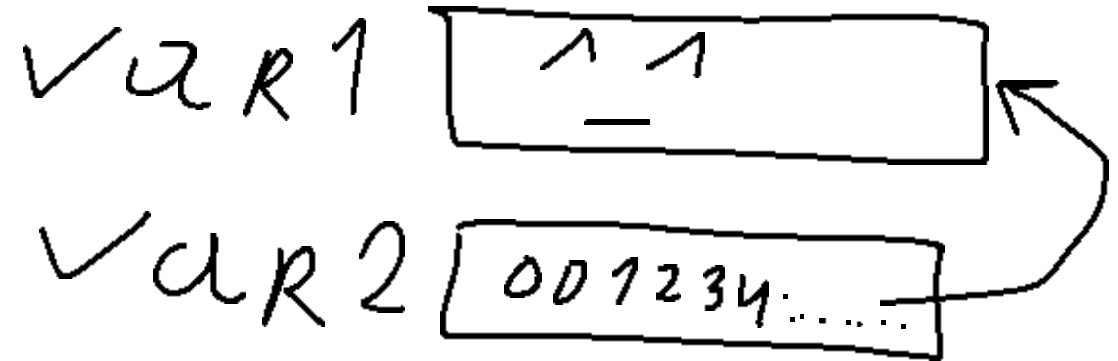
Borrowing aka Access via Reference

```
let var1 = my_struct;  
let var2 = &var1;  
println!("{}", var2);
```



Borrowing aka Access via Reference

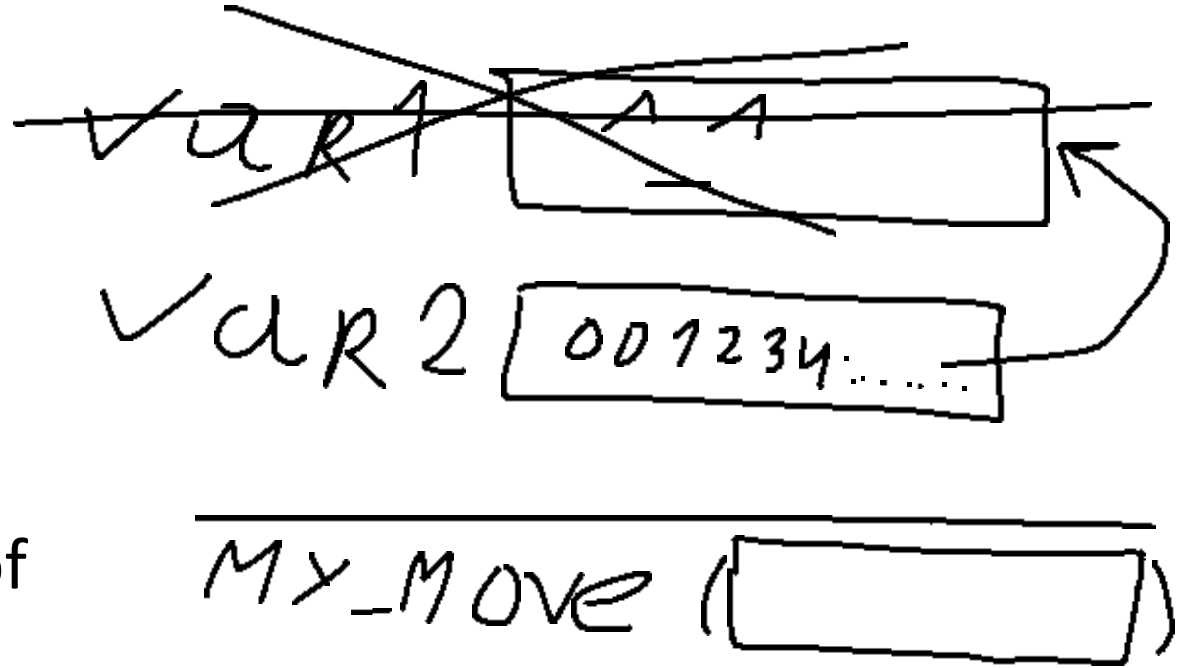
```
let var1 = my_struct;  
let var2 = &var1;  
println!("{}", var2);
```



Borrowing aka Access via Reference

```
let var1 = my_struct;  
let var2 = &var1;  
my_move(var1);  
println!("{}", var2);
```

error[E0505]: cannot move out of
`var1` because it is borrowed



Dangling References

```
fn create_and_return_ref() -> & String {  
    let s = String::from("Hi");  
    &s  
}
```

error[E0515]: cannot return reference to local variable `s`

&s

^^ returns a reference to data owned by the current function

Mutable References

```
let mut var = String::from("Hello");  
var.push_str(" World!");  
println!("{}", var)
```

Mutable References

```
let mut var = String::from("Hello");  
append_world(&var);  
println!("{}", var)
```

```
fn append_world(str: &String){  
    str.push_str(" World!")  
}
```

error[E0596]: cannot borrow `*str` as mutable, as it is behind a `&` reference

```
fn append_world(str: &String){
```

----- help: consider changing this to be a mutable reference: `&mut String`

```
str.push_str(" World!")
```

`str` is a `&` reference, so the data it refers to cannot be borrowed as mutable

Mutable References

```
let mut var = String::from("Hello");  
append_world(&var);  
println!("{}", var)
```

```
fn append_world(str: &mut String){  
    str.push_str(" World!")  
}
```

error[E0308]: mismatched types

```
append_world(&var);
```

----- ^^^^ types differ in mutability

Mutable References

```
let mut var = String::from("Hello");  
append_world(&mut var);  
println!("{}", var)
```

```
fn append_world(str: &mut String){  
    str.push_str(" World!")  
}
```

Mutable (Unique) vs Shared References

&mut – Mutable (Unique)

- Single
- No other Shared References
- Allow using in &mut parameters
- Allow using in & parameters

& - Shared

- Multiple
- No another Unique Reference
- Allow using in & parameters

Dereference via *

```
let x = 5;  
let y = &x;
```

```
assert_eq!(5, x);  
assert_eq!(5, *y);
```

Stealing Prevention for Drop Check

```
let var = String::from("Hi");  
let reference = &var;  
my_move(*reference);
```

```
fn my_move(my_struct: String) {}
```

error[E0507]: cannot move out of `\*reference` which is behind a shared reference

Assignment

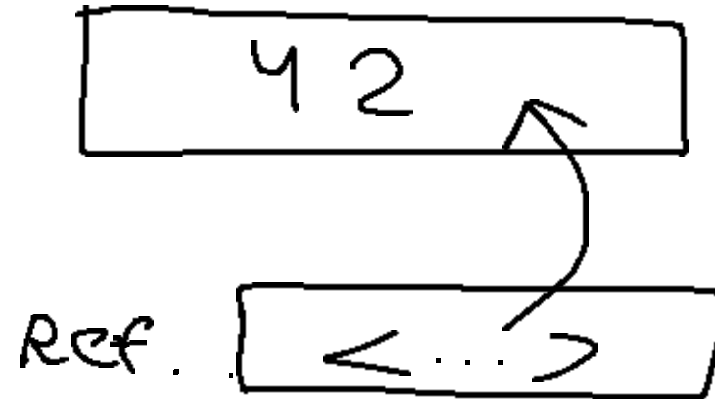
```
let mut var = String::from("Hi");  
let reference = &mut var;  
*reference = String::from("Lol");
```

Why is it ok?

Dereference move

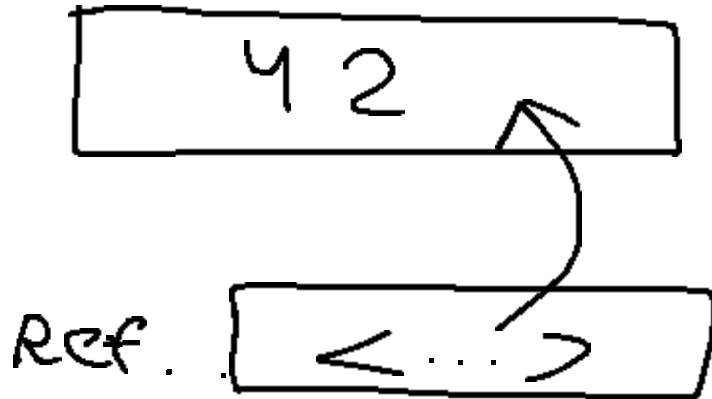
`let moved = *reference;`

`*reference = new_value;`

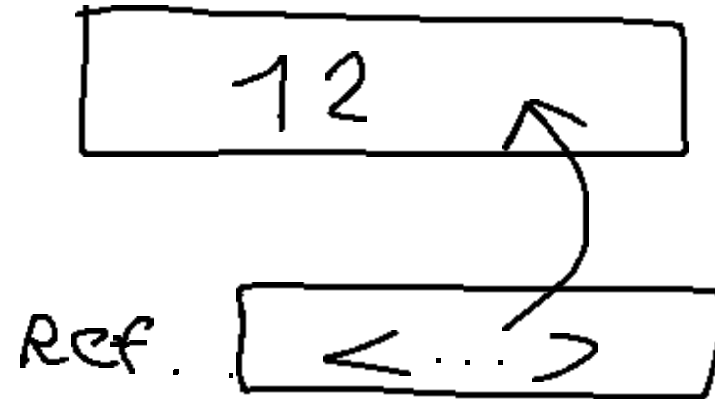


Dereference move

`let moved = *reference;`

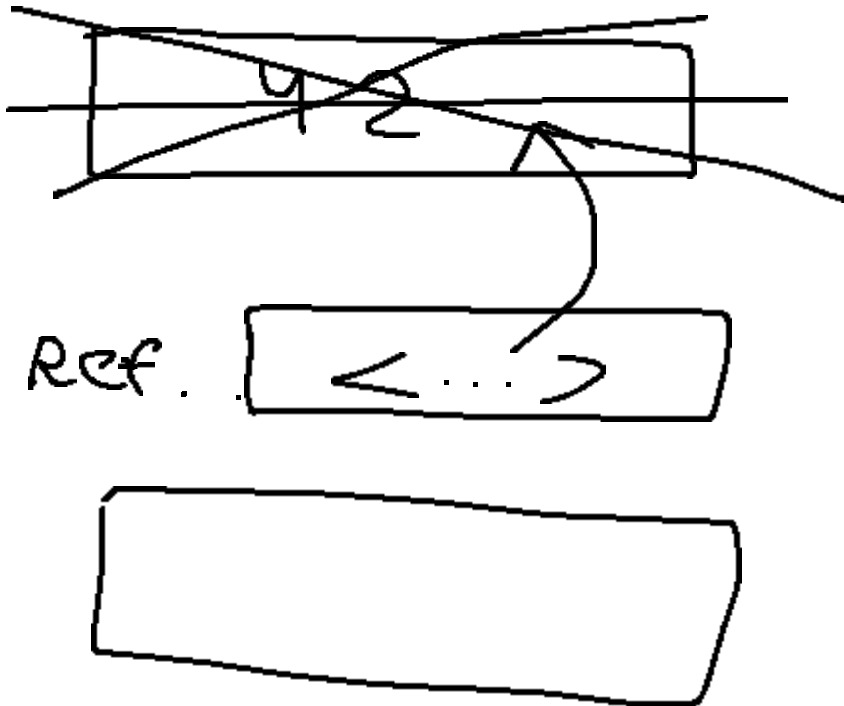


`*reference = new_value;`

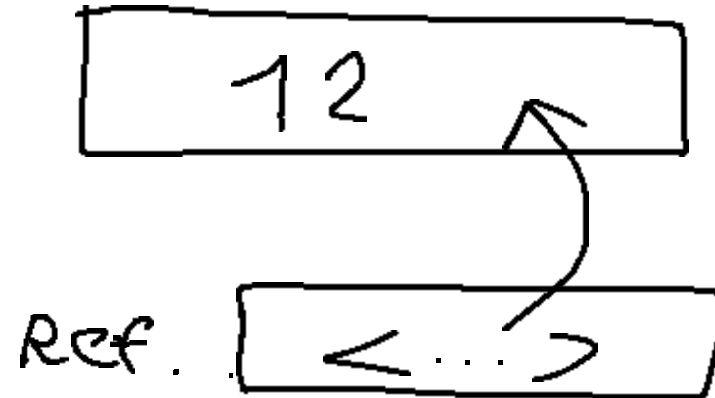


Dereference move

`let moved = *reference;`



`*reference = new_value;`



Dereference * as out

```
fn try_create(condition: bool, out: &mut String) -> bool {  
    if condition {  
        *out = String::from("Success");  
        return true;  
    }  
  
    return false;  
}
```

Dereference via .

```
let mut var = String::from("Hi");  
let reference = &mut var;
```

```
(*reference).push_str("Dude");
```

```
reference.push_str("Dude")
```

Stealing

```
let mut var = String::from("Hi");  
let reference = &mut var;  
let bytes = reference.into_bytes();
```

```
pub fn into_bytes(self) -> Vec<u8> {  
    self.vec  
}
```

error[E0507]: cannot move out of `\*reference` which is behind a mutable reference

Copy vs Move

```
let var = String::from("Hi");  
let reference = &var;  
let moved = var;  
println!("{}", reference)
```

error[E0505]: cannot move out of `var` because it is borrowed

```
let moved = var;
```

^^^ move out of `var` occurs here

```
println!("{}", reference)
```

----- borrow later used here

Copy vs Move

```
let var = 42;  
let reference = &var;  
let moved = var;  
println!("{}", reference)
```



Copy vs Move

Copy

```
let var = 42;  
let reference = &var;  
let copy = var;  
println!("{}", reference)
```

```
#[derive ! (Copy, Clone)]  
struct i32 {  
    ...  
}
```

Move

```
let var = String::from("Hi");  
let reference = &var;  
let moved = var;  
println!("{}", reference)
```

```
pub struct String {  
    ...  
}
```

Associated Functions

```
impl MyStruct {  
    fn any_static_method() {}  
  
    fn new(value1: i32, value2: bool) -> Self {  
        Self { value1, value2 }  
    }  
  
    fn print_me(&self) {  
        println!("{}", self.value1, self.value2);  
    }  
  
    fn update_me(&mut self, new_value1: i32) {  
        self.value1 = new_value1;  
    }  
  
    fn move_me(self, new_owner: &mut (i32, MyStruct)) {  
        new_owner.1 = self;  
    }  
  
    fn kill(self) {}  
}
```

```
MyStruct::any_static_method();
```

```
let mut str = MyStruct::new(42, true);
```

```
str.print_me();
```

```
str.update_me(666);
```

```
let mut new_owner = (42, MyStruct { value1: 0, value2:  
false });
```

```
str.move_me(&mut new_owner);
```

```
new_owner.1.print_me();
```

```
str.print_me(); <- value borrowed here after move
```

```
new_owner.1.kill();
```

Trait for everything

```
trait MyTrait {  
    fn do (self);  
}
```

```
impl MyTrait for &MyStruct {  
    fn do (self) {  
        let it_is_reference: &MyStruct = self;  
    }  
}
```

Variable/Field State

	Can Move	Can Borrow	Can Borrow Mut	Can Reassign
<code>let mut str = MyStruct { int: 42 };</code>	True	True	True	True
<code>let shared_ref = &str;</code>	False	True	False	False
<code>let mut_ref = &mut str;</code>	False	False	False	False
<code>drop(str);</code>	False	False	False	True

Lifetimes

Lifetimes are **named regions of code** that a reference must be valid for.

- Compile Time only
- Drop Check relies on Lifetimes
- Elided if necessary

Lifetimes

```
{ // lifetime: 'a
  let var1: i32 = 42;
  { // lifetime: 'b
    let var2: &'b i32 = &'b var1;
  }
}
```

```
let var1: i32 = 42;
let var2: &i32 = &var1;
```

Lifetimes

```
{ // lifetime: 'a
  let var1: i32 = 42;
  { // lifetime: 'b
    let var2: &'b i32 = &'b var1;
    { // lifetime: 'c
      let var3: &'c &'b i32 = &'c
var2;
    }
  }
}
```

```
let var1: i32 = 42;
let var2: &i32 = &var1;
let var3: &&i32 = &var2;
```


Lifetimes

```
{ // lifetime: 'a
  let var: &i32;
  { // lifetime: 'b
    let var1: i32 = 42;
    { // lifetime: 'c ←
      let var2: &'a i32 = &'a var1;
      var = var2;
    }
  }
}
```

```
let var: &i32;
```

```
let var1: i32 = 42;
```

```
let var2: &i32 = &var1;
```

```
let var3: &&i32 = &var2;
```

```
var = var2;
```

Lifetimes and Dangling Pointer

```
fn create_and_return_ref<'x>() -> &'x String {  
    { // lifetime 'b  
        let s = String::from("Hi");  
        &'x s  
    }  
}
```

```
fn caller_dangling_string(){  
    { // lifetime 'x  
        let string_ref: &'x String = create_and_return_ref();  
    }  
}
```

Lifetimes and Dangling Pointer

```
fn caller_dangling_string() {  
    { // lifetime 'x'  
        let string_ref: &'x String;  
        { // lifetime 'b'  
            let s = String::from("Hi");  
            string_ref = &'xs ?  
        }  
    }  
}
```

Lifetimes and Dangling Pointer

```
fn create_and_return_ref(my_struct: &mut MyStruct) -> &String {  
    my_struct.str = String::from("Hi");  
    &my_struct.str  
}
```

```
fn caller_non_dangling_string() {  
    let mut my_struct = MyStruct { str: String::default() };  
    let string_ref: &String = create_and_return_ref(&mut my_struct);  
    print!("{}", string_ref)  
}
```

How to avoid Dangling Pointer

```
{// lifetime 'a
  let mut my_struct = MyStruct { str: String::default() };
  let string_ref: &'a String;
  { // lifetime 'b
    my_struct.str = String::from("Hi");
  }
  string_ref = &'a my_struct.str;
  print!("{}", string_ref)
}
```

How to avoid Dangling Pointer and LEAK

```
fn create_and_return_ref() -> &'static String {  
    let s = Box::leak(Box::new(String::from("Hi")));  
    s  
}
```

Static lifetime: Lives forever

```
fn static_instance() {  
    let val = String::from("Hi");  
    let reference: &'static String = &val;  
}
```

error[E0597]: `val` does not live long enough

```
let reference: &'static String = &val;
```

^^^^ borrowed value does not live long enough

```
}
```

- `val` dropped here while still borrowed

How mut check works

```
{ // lifetime 'a
  let mut var: String = String::from("Hello");
  { // lifetime 'b
    let reference: &'b String = &'b var;
    { // lifetime 'c
      let mut_reference: &'c mut String =
        &'b mut var;
      mut_reference.push_str("World");
    }
    println!("{}", reference)
  }
}
```

```
let mut var = String::from("Hello");
let reference = &var;
let mut_reference = &mut var;
mut_reference.push_str("World");
println!("{}", reference)
```


Lifetimes that you have to specify

```
fn create (s1: &mut MyStruct, s2: &mut MyStruct) -> &String {  
    s1.str = String::from("Hi");  
    &s1.str  
}
```

error[E0106]: missing lifetime specifier

```
fn create(s1: &mut MyStruct, s2: &mut MyStruct) -> &String {  
      
    expected named lifetime parameter ^
```

Compiler POV

```
fn create(s1: &mut MyStruct, s2: &mut MyStruct) -> &String {  
    s1.str = String::from("Hi");  
    &s1.str  
}  
{ // lifetime 'a  
    let mut my_struct1 = MyStruct { str: String::default() };  
    { // lifetime 'b  
        let mut my_struct2 = MyStruct { str: String::default() };  
        let string: &String = create(&mut my_struct1, &mut my_struct2);  
        println!("{}", string);  
    }  
}
```

Compiler POV

```
fn create (s1: &mut MyStruct, s2: &mut MyStruct) -> &String {  
    s1.str = String::from("Hi");  
    &s1.str  
}  
{ // lifetime 'a  
    let mut my_struct1 = MyStruct { str: String::default() };  
    { // lifetime 'b  
        let mut my_struct2 = MyStruct { str: String::default() };  
        let string: &'? String = create(&'a mut my_struct1, &'b mut my_struct2);  
        println!("{}", string);  
    }  
}
```

Lifetimes that elision

```
fn create<'a>(s1: &'a mut MyStruct, s2: &mut MyStruct) -> &'a String {  
    s1.str = String::from("Hi");  
    &s1.str  
}  
  
{ // lifetime 'a  
    let mut my_struct1 = MyStruct { str: String::default() };  
    { // lifetime 'b  
        let mut my_struct2 = MyStruct { str: String::default() };  
        let string: &'a String = create(&'a mut my_struct1, &'b mut my_struct2);  
        println!("{}", string);  
    }  
}
```

Structs with References

```
struct MyStructWithRef<'a, 'b> {  
    reference1: &'a String,  
    reference2: &'b String,  
}
```

```
let val1 = String::from("Foo");  
let val2 = String::from("Bar");
```

```
let str = MyStructWithRef {  
    reference1: &val1,  
    reference2: &val2,  
};
```

Structs with References: Why???

```
struct MyStructWithRef<'a, 'b> {  
    reference1: &'a String,  
    reference2: &'b String,  
}
```

```
fn create_struct_with_ref() -> MyStructWithRef {  
    let val1 = String::from("Foo");  
    let val2 = String::from("Bar");  
    let str = MyStructWithRef {  
        reference1: &val1,  
        reference2: &val2,  
    };  
    return str;  
}
```

Structs with References

```
fn create_struct_with_ref<'a>() -> MyStructWithRef<'a, 'a> {  
    let val1 = String::from("Foo");  
    let val2 = String::from("Bar");  
    let str = MyStructWithRef {  
        reference1: &val1,  
        reference2: &val2,  
    };  
    return str;  
}
```

^^^ returns a value referencing data owned by the current function

Structs with References

```
fn main() {  
    let mut val1 = String::new();  
    let mut val2 = String::new();  
    let str = create_struct_from_strings(&mut val1, &mut val2);  
}  
  
fn create_struct_from_strings<'a>(val1: &'a mut String, val2: &'a mut String)  
    -> MyStructWithRef<'a, 'a> {  
    *val1 = String::from("Foo");  
    *val2 = String::from("Bar");  
    let str = MyStructWithRef {  
        reference1: val1,  
        reference2: val2,  
    };  
    return str;  
}
```


Return Structs with References vs &

```
fn create<'a>(s1: &'a mut MyStruct, s2: &mut MyStruct) -> &'a String
```

vs

```
fn create_struct_from_strings<'a>(val1: &'a mut String, val2: &'a mut String) -> MyStructWithRef<'a, 'a>
```

```
struct MyStructWithRef<'a, 'b> {  
    reference1: &'a String,  
    reference2: &'b String,  
}
```

Multiple lifetimes in structs

```
struct MyStructWithRef<'a> {  
    reference1: &'a String,  
    reference2: &'a String,  
}
```

```
let val1 = String::from("Foo");  
let reference2: &'static mut String = Box::leak(Box::new(String::from("Hi")));
```

```
let str = MyStructWithRef {  
    reference1: &val1,  
    reference2: reference2,  
};
```

← borrowed value does not live long enough

```
let r: &'static String = str.reference2;
```

Multiple lifetimes in structs

```
struct MyStructWithRef<'a, 'b> {  
    reference1: &'a String,  
    reference2: &'b String,  
}
```

```
let val1 = String::from("Foo");  
let reference2: &'static mut String = Box::leak(Box::new(String::from("Hi")));
```

```
let str = MyStructWithRef {  
    reference1: &val1,  
    reference2: reference2,  
};
```

```
let r: &'static String = str.reference2;
```

Copy, Clone for &mut

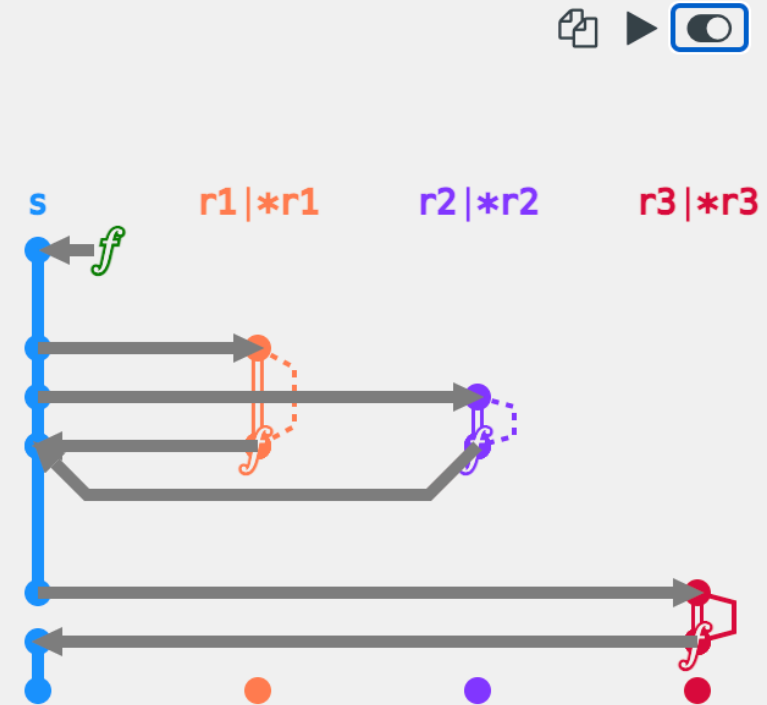
```
#[derive(Copy, Clone)]  
struct MyReference<'a> {  
    reference: &'a mut MyStr,  
}
```

error[E0204]: the trait `Copy` cannot be implemented for this type

<https://github.com/rustviz/rustviz>

Hover over timeline events (dots), states (vertical lines), and actions (arrows) for extra information.

```
1 fn main(){
2     let mut s = String::from("hello");
3
4     let r1 = &s;
5     let r2 = &s;
6     assert!(compare_strings(r1, r2));
7
8     let r3 = &mut s;
9     clear_string(r3);
10 }
```



Homework

- Rustlings
 - lifetimes
 - move_semantics
- Courseware:
 - Refactoring: use `&`, `&mut` to avoid extra memory overhead